第 5 章 图论基础

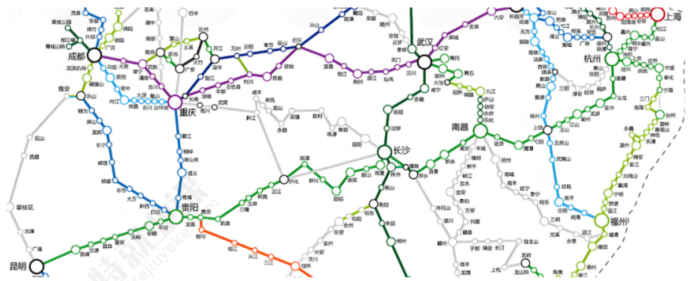
1. 图的基本概念

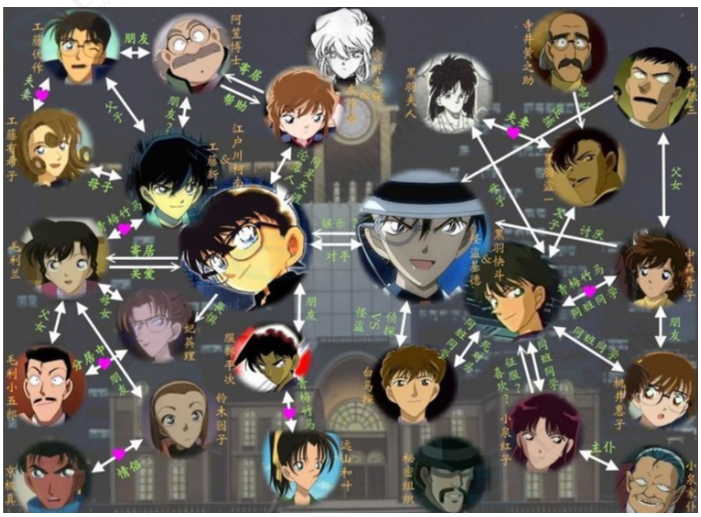
1.1 图的定义

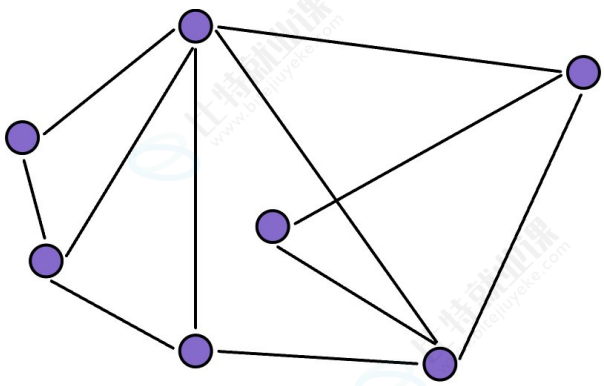
> 图 G 是由顶点集 V 和边集 E 组成，记为 $G=(V,E)$，其中 $V(G)$ 表示图 G 中顶点的有限非空集；$E(G)$ 表示图 G 中顶点之间的关系（边）集合。若 $V=\{v_1,v_2,\dots,v_n\}$，则用 $|V|$ 表示图 G 中顶点的个数，也称图 G 的阶，$E=\{(u,v)|u\in V,v\in V\}$，用 $|E|$ 表示图 G 中边的条数。

图是较线性表和树更为复杂的数据结构。

- 线性表中，除第一个和最后一个元素外，每个元素只有一个唯一的前驱和唯一的后继结点，元素和元素之间是一对一的关系；
- 在树形结构中，数据元素之间有着明显的层次关系，每个元素有唯一的双亲，但可能有多个孩子，元素和元素之间是一对多的关系；
- 而图形结构中，元素和元素之间的关系更加复杂，结点和结点之间的关系是任意的，任意两个结点之间都可能相关，图中元素和元素之间是多对多的关系。



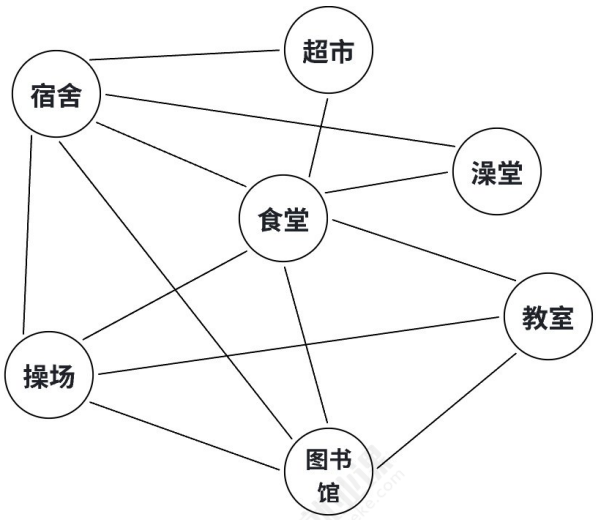




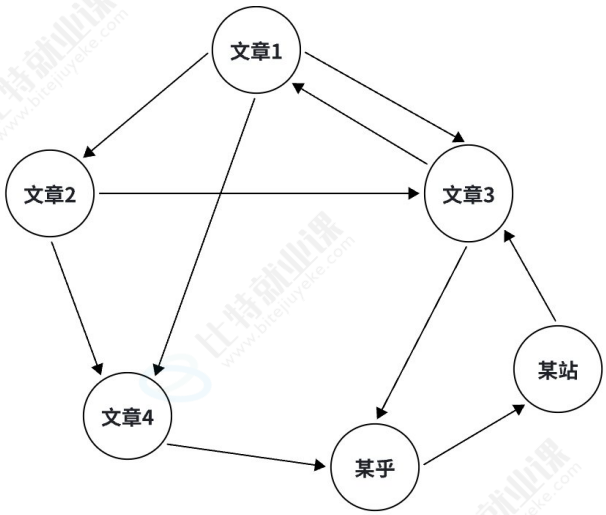
图结构

1.2 有向图和无向图

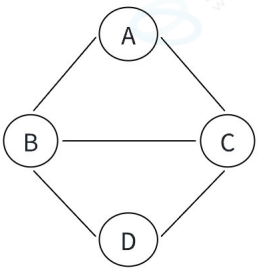
图根据边的类型，可以分为无向图和有向图：



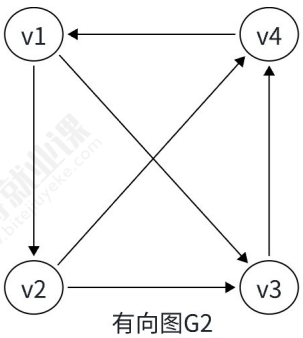
校园方位图【无向图】
校内的各个地方为顶点
两个地点之间的道路为边
两个地方间的边没有方向



网页链接图【有向图】
网页可以看作顶点
网页间的链接关系看作边
网页中指向其他网页的链接有方向

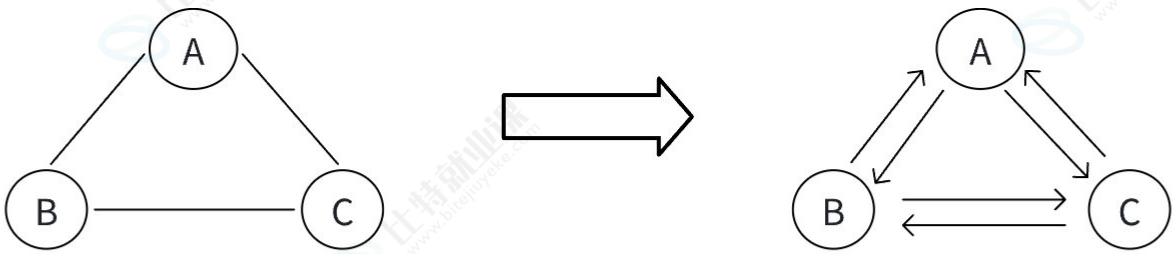


无向图G1



有向图G2

在图相关的算法中，我们可以将无向图中的边看成两条方向相反的有向边，从而将无向图转化为有向图：

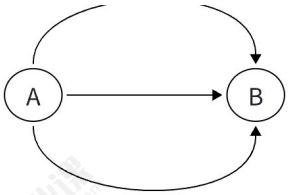


1.3 简单图与多重图

自环：自己指向自己的一条边。

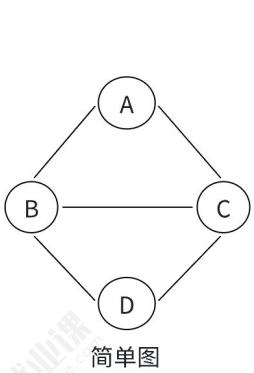


重边：图中存在两个或两个以上完全相同的边。

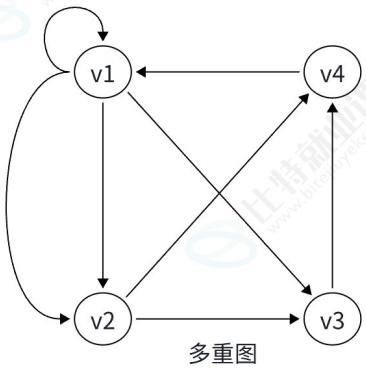


简单图：若图中没有重边和自环，为简单图。

多重图：若图中存在重边或自环，为多重图。



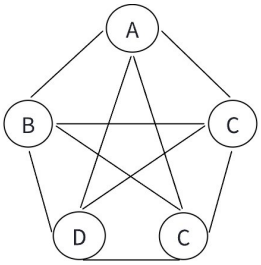
简单图



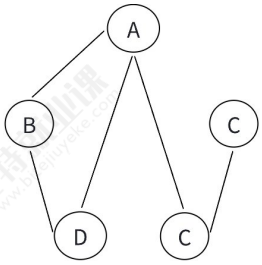
多重图

1.4 稠密图和稀疏图

有很少条边(如 $e < nlog_2 n$)的图称为稀疏图，反之称为稠密图。



稠密图

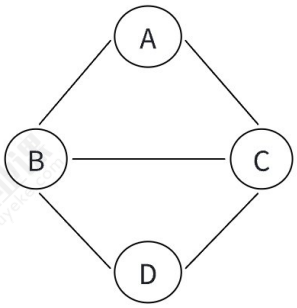


稀疏图

1.5 顶点的度

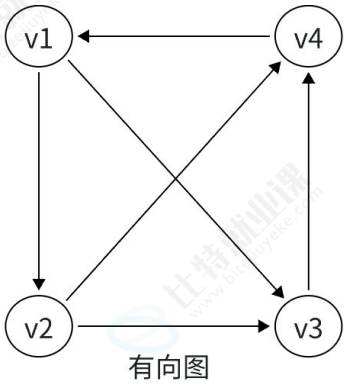
顶点 v 的度是指与它相关联的边的条数，记作 deg(v)。由该顶点发出的边称为顶点的出度，到达该顶点的边称为顶点的入度。

- 无向图中，顶点的度等于该顶点的入度(indev)和出度(outdev)，即 deg(v) = indeg(v) = outdeg(v)。
- 有向图中，顶点的度等于该顶点的入度与出度之和，其中顶点 v 的入度 indeg(v) 是以 v 为终点的有向边的条数，顶点 v 的出度 outdeg(v) 是以 v 为起始点的有向边的条数，deg(v) = indeg(v) + outdeg(v)。



无向图

dev(B) = indev(B) = outdev(B) = 3
无向图中所有顶点的度之和为总边数的2倍



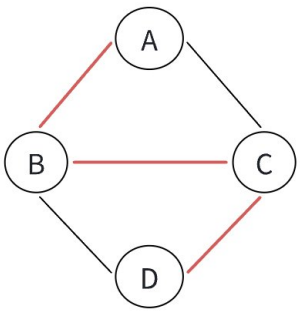
有向图

顶点v1的入度为：indev(v1) = 1
顶点v1的出度为：outdev(v1) = 2
顶点v1的度为：dev(v1) = indev(v1) + outdev(v1) = 3

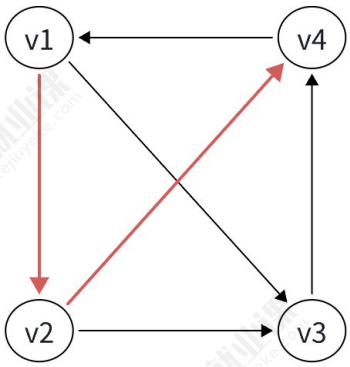
1.6 路径

在图 G = (V， E) 中，若从顶点 v_i 出发，沿一些边经过某些顶点 $v_{p1}, v_{p2}, \ldots, v_{pm}$ ，到达顶点 v_j 。则称顶点序列 ($v_i, v_{p1}, v_{p2}, \ldots, v_{pm}, v_j$) 为从顶点 v_i 到顶点 v_j 的路径。

注意：两个顶点间的路径可能不唯一。



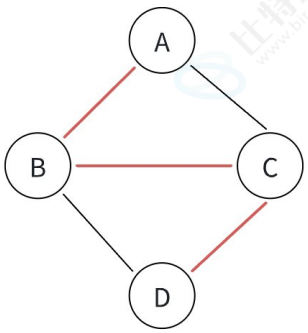
顶点A到顶点D的一条路径



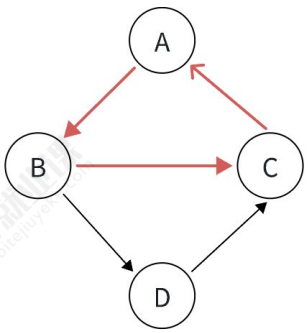
顶点v1到顶点v4的一条路径

1.7 简单路径与回路

若路径上各顶点 $v_1, v_2, v_3, \ldots, v_m$ 均不重复，则称这样的路径为简单路径。若路径上第一个顶点 v_1 和最后一个顶点 v_m 相同，则称这样的路径为回路或环。



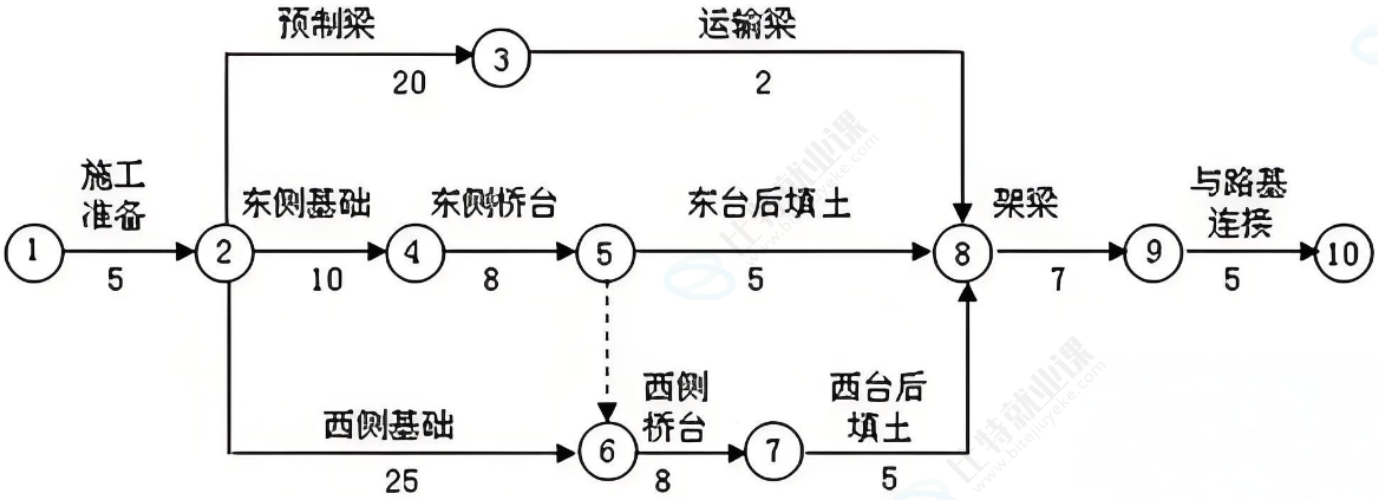
A->B->D的简单路径



A->B-D-A的回路(或环)

1.8 路径长度和带权路径长度

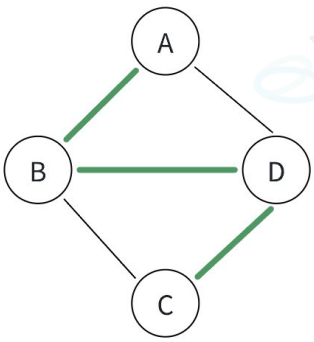
某些图的边具有与它相关的数值，称其为该边的**权值**。这些权值可以表示两个顶点间的距离、花费的代价、所需的时间等。一般将该种带权图称为网络。



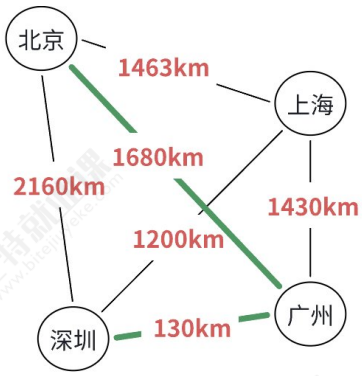
某工程施工周期图

对于**不带权的图，一条路径的路径长度是指该路径上的边的条数。**

对于**带权的图，一条路径的路径长度是指该路径上各个边权值的总和。**



从A到C的路径中包含3条边
路径长度为3

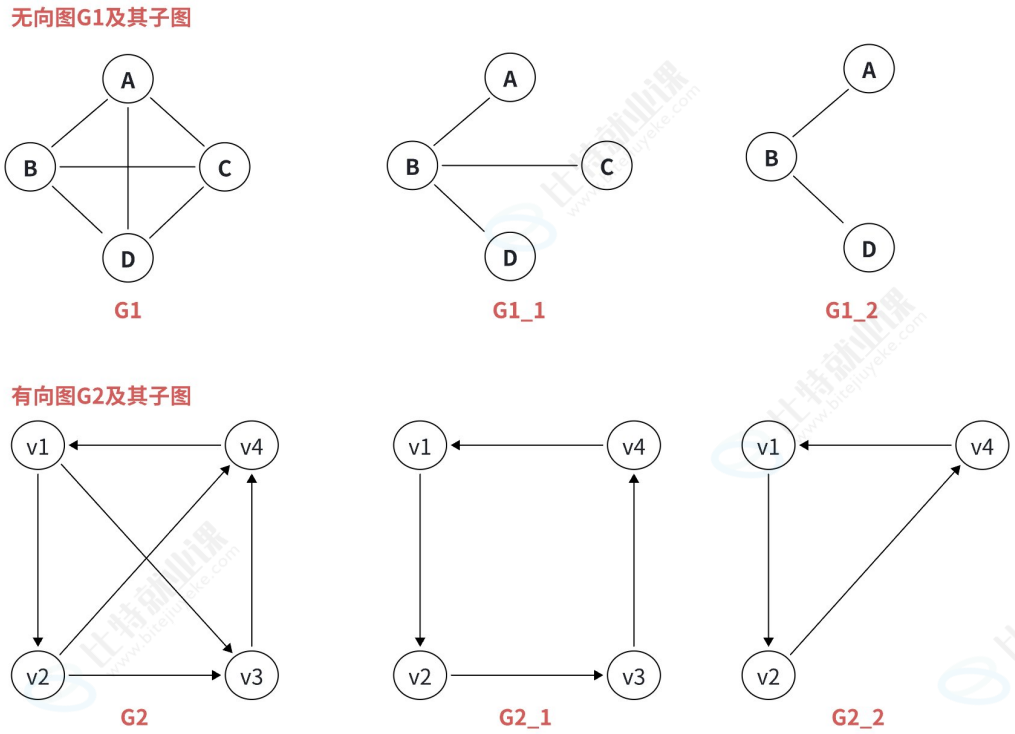


从北京到深圳的带权路径长度为：
1680km + 130km = 1810km

1.9 子图

设图 $G=\{V,E\}$ 和图 $G'=\{V',E'\}$，若 $V'\in V$ 且 E' 属于 E，则称 G' 是 G 的子图。若有 $V(G')=V(G)$ 的子图 G'，则称 G' 为 G 的生成子图。

相当于就是在原来图的基础上，拿出来一些顶点和边，组成一个新的图。但是要注意，拿出来的点和边要能构成一个图才行。



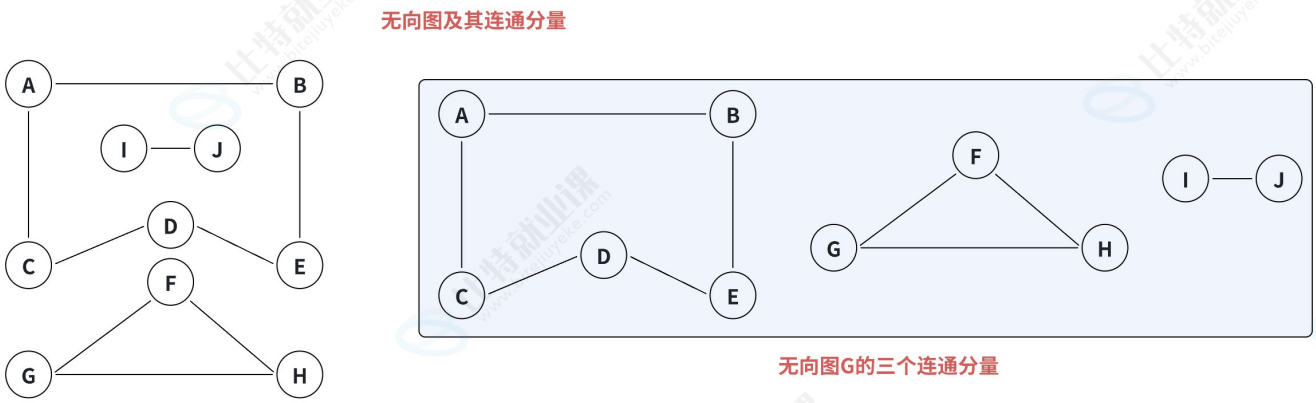
G1_1 和 G1_2 为无向图 G1 的子图，G1_1 为 G1 的生成子图。

G2_1 和 G2_2 为有向图 G2 的子图，G2_1 为 G2 的生成子图。

1.10 连通图与连通分量

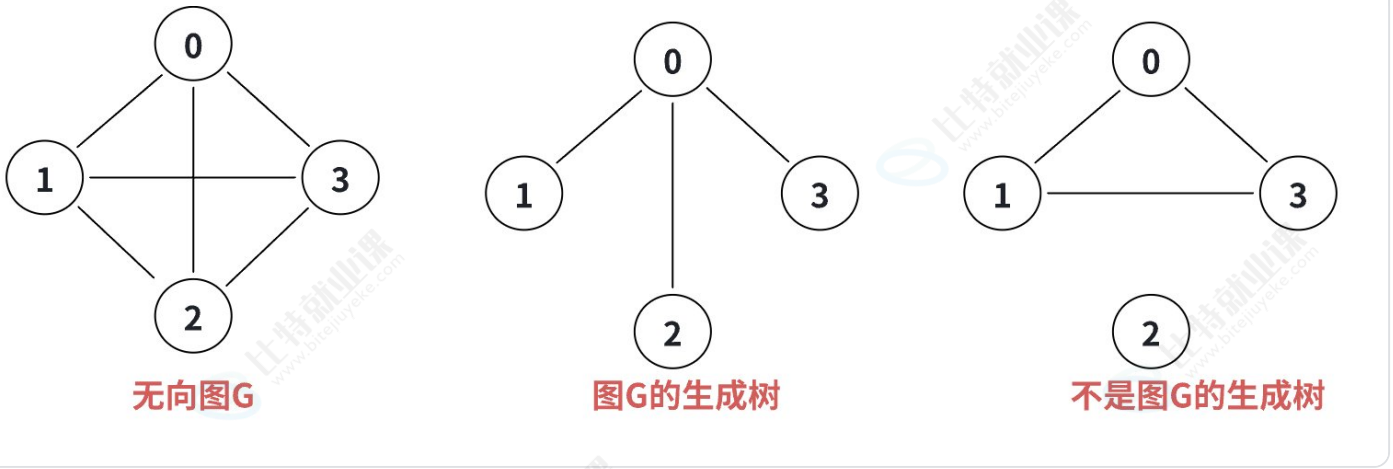
在**无向图**中，若从顶点 v_1 到顶点 v_2 有路径，则称顶点 v_1 与顶点 v_2 是连通的。**如果图 G 中任意一对顶点都是连通的，则图 G 称为连通图，否则称为非连通图。**

- **假设一个图有 n 个顶点，如果边数小于 n - 1，那么此图一定是非连通图。**
- **极大联通子图：无向图中，拿出一个子图，这个子图包含尽可能多的点和边。**
- **连通分量：无向图中的极大连通子图称为连通分量。**



1.11 生成树

连通图的生成树是包含图中全部顶点的一个极小连通子图。若图中顶点数为 n，则它的生成树含有 n - 1 条边。对生成树而言，若砍去一条边，则会变成非连通图，若加上一条边则会形成一个回路。



2. 图的存储和遍历

图的存储有两种：邻接矩阵和邻接表：

- 其中，邻接表的存储方式与树的孩子表示法完全一样。因此，用 vector 数组以及链式前向星就能实现。
- 而邻接矩阵就是用一个二维数组，其中 `edges[i][j]` 存储顶点 `i` 与顶点 `j` 之间，边的信息。

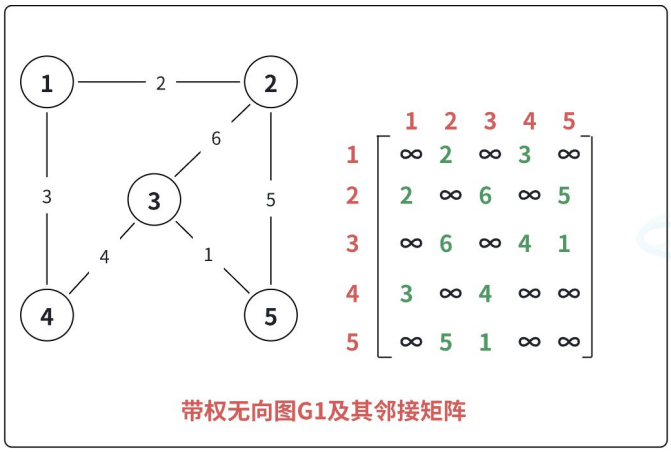
图的遍历分两种：DFS 和 BFS，和树的遍历方式以及实现方式完全一样。因此，可以仿照树这个数据结构来学习。

2.1 邻接矩阵

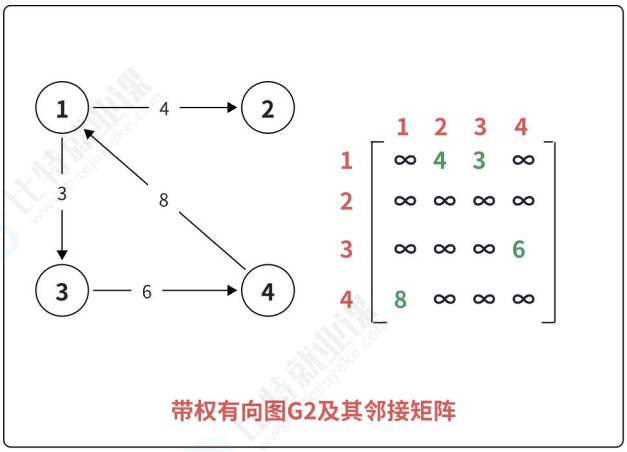
邻接矩阵，指用一个矩阵(即二维数组)存储图中边的信息(即各个顶点之间的邻接关系)，存储顶点之间邻接关系的矩阵称为邻接矩阵。

对于带权图而言，若顶点 v_i 和 v_j 之间有边相连，则邻接矩阵中对应项存放着该边对应的权值，若顶点 v_i 和 v_j 不相连，则用 ∞ 来代表这两个顶点之间不存在边。

对于不带权的图，可以创建一个二维的 bool 类型的数组，来标记顶点 v_i 和 v_j 之间有边相连。



带权无向图G1及其邻接矩阵



带权有向图G2及其邻接矩阵

【注意】

矩阵中元素个数为 $n \times n$，即空间复杂度为 $O(n^2)$，n 为顶点个数，和实际边的条数无关，**适合存储稠密图**。

代码实现：

```cpp
#include <iostream>
#include <cstring>

using namespace std;

const int N = 1010;

int n, m;
int edges[N][N];

int main()
{
    memset(edges, -1, sizeof edges);

    cin >> n >> m; // 读入结点个数以及边的个数
    for(int i = 1; i <= m; i++)
    {
        int a, b, c; cin >> a >> b >> c;
        // a - b 有一条边，权值为 c
        edges[a][b] = c;

        // 如果是无向边，需要反过来再存一下
        edges[b][a] = c;
    }

```

```
    return 0;
}
```

2.2 vector 数组

和树的存储一模一样，只不过如果存在边权的话，我们的 vector 数组里面放一个结构体或者是 pair 即可。

代码实现：

```
#include <iostream>
#include <vector>

using namespace std;

typedef pair<int, int> PII;

const int N = 1e5 + 10;

int n, m;
vector<PII> edges[N];

int main()
{
    cin >> n >> m; // 读入结点个数以及边的个数
    for(int i = 1; i <= m; i++)
    {
        int a, b, c; cin >> a >> b >> c;
        // a 和 b 之间有一条边，权值为 c
        edges[a].push_back({b, c});

        // 如果是无向边，需要反过来再存一下
        edges[b].push_back({a, c});
    }

    return 0;
}
```

2.3 链式前向星

和树的存储一模一样，只不过如果存在边权的话，我们多创建一维数组，用来存储边的权值即可。

代码实现：

```cpp
#include <iostream>

using namespace std;

const int N = 1e5 + 10;

// 链式前向星
int h[N], e[N * 2], ne[N * 2], w[N * 2], id;
int n, m;

// 其实就是把 b 头插到 a 所在的链表后面
void add(int a, int b, int c)
{
    id++;
    e[id] = b;
    w[id] = c; // 多存一个权值信息
    ne[id] = h[a];
    h[a] = id;
}

int main()
{
   cin >> n >> m; // 读入结点个数以及边的个数
    for(int i = 1; i <= m; i++)
    {
        int a, b, c; cin >> a >> b >> c;
        // a 和 b 之间有一条边，权值为 c
        add(a, b, c); add(b, a, c);
    }

    return 0;
}
```

2.4 DFS

和树的遍历方式一模一样，一条路走到黑~

1. 用邻接矩阵的方式存储

代码实现：

```cpp
#include <iostream>
#include <cstring>
#include <queue>

using namespace std;

const int N = 1010;

int n, m;
int edges[N][N];

bool st[N]; // 标记哪些点已经访问过

void dfs(int u)
{
    cout << u << endl;
    st[u] = true;

    // 遍历所有孩子
    for(int v = 1; v <= n; v++)
    {
         // 如果存在 u->v 的边，并且没有遍历过
        if(edges[u][v] != -1 && !st[v])
        {
            dfs(v);
        }
    }
}

int main()
{
    memset(edges, -1, sizeof edges);

    cin >> n >> m; // 读入结点个数以及边的个数
    for(int i = 1; i <= m; i++)
    {
        int a, b, c; cin >> a >> b >> c;
        // a - b 有一条边，权值为 c
        edges[a][b] = c;

        // 如果是无向边，需要反过来再存一下
        edges[b][a] = c;
    }
```

```
44
45      return 0;
46  }
```

2. 用 vector 数组的方式存储

代码实现：

```
1   #include <iostream>
2   #include <vector>
3   #include <queue>
4
5   using namespace std;
6
7   typedef pair<int, int> PII;
8
9   const int N = 1e5 + 10;
10
11  int n, m;
12  vector<PII> edges[N];
13
14  bool st[N]; // 标记哪些点已经访问过
15
16  void dfs(int u)
17  {
18      cout << u << endl;
19      st[u] = true;
20
21      // 遍历所有孩子
22      for(auto& t : edges[u])
23      {
24          // u->v 的一条边，权值为 w
25          int v = t.first, w = t.second;
26          if(!st[v])
27          {
28              dfs(v);
29          }
30      }
31  }
32
33  int main()
34  {
35      cin >> n >> m; // 读入结点个数以及边的个数
36      for(int i = 1; i <= m; i++)
```

```
    {
        int a, b, c; cin >> a >> b >> c;
        // a 和 b 之间有一条边，权值为 c
        edges[a].push_back({b, c});

        // 如果是无向边，需要反过来再存一下
        edges[b].push_back({a, c});
    }

    return 0;
}
```

3. 用链式前向星的方式存储

代码实现：

```
#include <iostream>
#include <queue>

using namespace std;

const int N = 1e5 + 10;

// 链式前向星
int h[N], e[N * 2], ne[N * 2], w[N * 2], id;
int n, m;

// 其实就是把 b 头插到 a 所在的链表后面
void add(int a, int b, int c)
{
    id++;
    e[id] = b;
    w[id] = c; // 多存一个权值信息
    ne[id] = h[a];
    h[a] = id;
}

bool st[N];

void dfs(int u)
{
    cout << u << endl;
    st[u] = true;

```

```
    // 遍历所有的孩子
    for(int i = h[u]; i; i = ne[i])
    {
        // u->v 的一条边
        int v = e[i];
        if(!st[v])
        {
            dfs(v);
        }
    }
}

int main()
{
   cin >> n >> m; // 读入结点个数以及边的个数
    for(int i = 1; i <= m; i++)
    {
        int a, b, c; cin >> a >> b >> c;
        // a 和 b 之间有一条边，权值为 c
        add(a, b, c); add(b, a, c);
    }

    return 0;
}
```

2.5 BFS

和树的遍历方式一模一样，一层一层的剥开我的心~

1. 用邻接矩阵的方式存储

代码实现：

```
#include <iostream>
#include <cstring>
#include <queue>

using namespace std;

const int N = 1010;

int n, m;
int edges[N][N];
```

```cpp

bool st[N]; // 标记哪些点已经访问过

void bfs(int u)
{
    queue<int> q;
    q.push(u);
    st[u] = true;

    while(q.size())
    {
        auto a = q.front(); q.pop();
        cout << a << endl;

        for(int b = 1; b <= n; b++)
        {
            if(edges[a][b] != -1 && !st[b])
            {
                q.push(b);
                st[b] = true;
            }
        }
    }
}

int main()
{
    memset(edges, -1, sizeof edges);

    cin >> n >> m; // 读入结点个数以及边的个数
    for(int i = 1; i <= m; i++)
    {
        int a, b, c; cin >> a >> b >> c;
        // a - b 有一条边，权值为 c
        edges[a][b] = c;

        // 如果是无向边，需要反过来再存一下
        edges[b][a] = c;
    }

    return 0;
}
```

2. 用 vector 数组的方式存储

代码实现：

```cpp
#include <iostream>
#include <vector>
#include <queue>

using namespace std;

typedef pair<int, int> PII;

const int N = 1e5 + 10;

int n, m;
vector<PII> edges[N];

bool st[N]; // 标记哪些点已经访问过

void bfs(int u)
{
    queue<int> q;
    q.push(u);
    st[u] = true;

    while(q.size())
    {
        auto a = q.front(); q.pop();
        cout << a << endl;

        for(auto& t : edges[a])
        {
            int b = t.first, c = t.second;
            if(!st[b])
            {
                q.push(b);
                st[b] = true;
            }
        }
    }
}

int main()
{
    cin >> n >> m; // 读入结点个数以及边的个数
    for(int i = 1; i <= m; i++)
    {
        int a, b, c; cin >> a >> b >> c;
```

```
        // a 和 b 之间有一条边，权值为 c
        edges[a].push_back({b, c});

        // 如果是无向边，需要反过来再存一下
        edges[b].push_back({a, c});
    }

    return 0;
}
```

3. 用链式前向星的方式存储

代码实现：

```
#include <iostream>
#include <queue>

using namespace std;

const int N = 1e5 + 10;

// 链式前向星
int h[N], e[N * 2], ne[N * 2], w[N * 2], id;
int n, m;

// 其实就是把 b 头插到 a 所在的链表后面
void add(int a, int b, int c)
{
    id++;
    e[id] = b;
    w[id] = c; // 多存一个权值信息
    ne[id] = h[a];
    h[a] = id;
}

bool st[N];

void bfs(int u)
{
    queue<int> q;
    q.push(u);
    st[u] = true;

    while(q.size())
```

```
    {
        auto a = q.front(); q.pop();
        cout << a << endl;

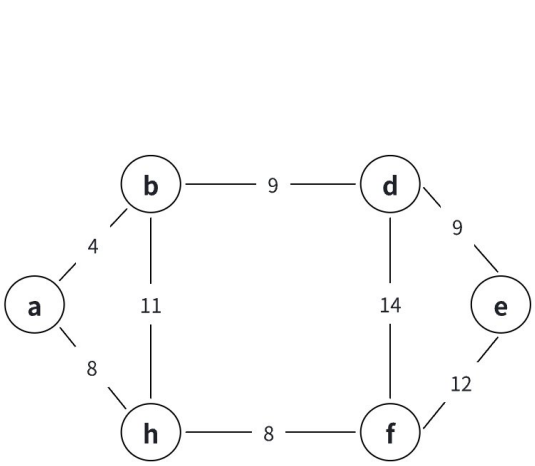
        for(int i = h[a]; i; i = ne[i])
        {
            int b = e[i], c = w[i];
            if(!st[b])
            {
                q.push(b);
                st[b] = true;
            }
        }
    }
}

int main()
{
   cin >> n >> m; // 读入结点个数以及边的个数
    for(int i = 1; i <= m; i++)
    {
        int a, b, c; cin >> a >> b >> c;
        // a 和 b 之间有一条边，权值为 c
        add(a, b, c); add(b, a, c);
    }

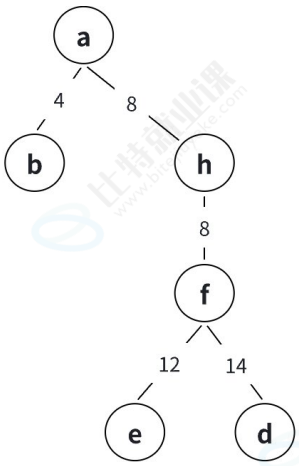
    return 0;
}
```

3. 最小生成树

通过前文的学习知道，一个具有 n 个顶点的连通图，其生成树为包含 $n-1$ 条边和所有顶点的极小连通子图。对于生成树来说，若砍去一条边就会使图不连通图；若增加一条边就会形成回路。

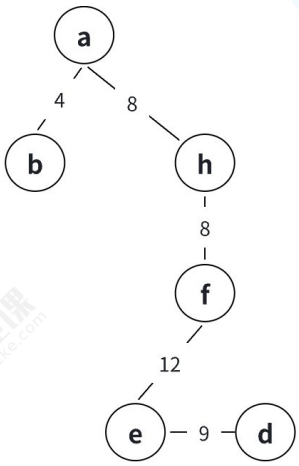


图G



图G的生成树t1

权值之和：4+8+8+12+14=46



图G的生成树t2

所有边权值之和：4+8+8+12+9=41

一个图的生成树可能有多个，将所有生成树中权值之和最小的树称为最小生成树。

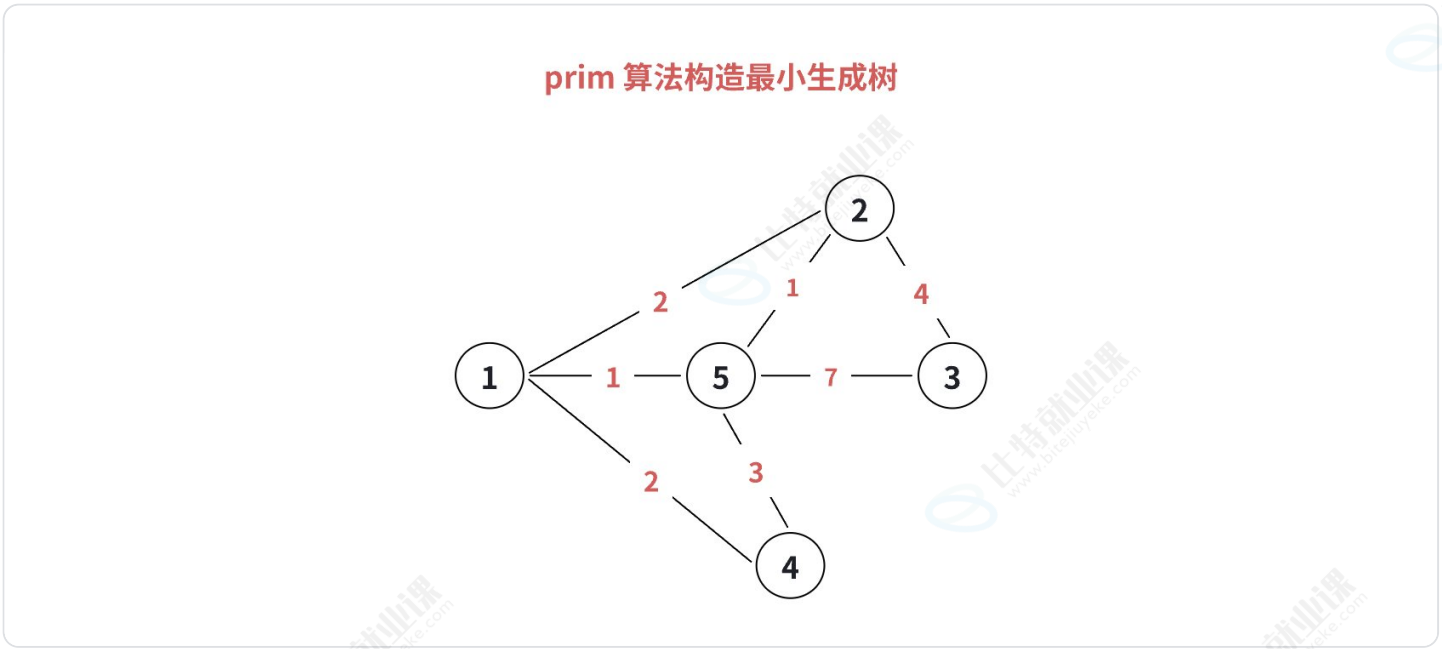
构造最小生成树有多种算法，典型的有普利姆 (Prim) 算法和克鲁斯卡尔 (Kruskal) 算法两种，它们都是基于贪心的策略。下面讲解算法的具体流程，关于正确性的证明，有兴趣的同学可以看看《算法导论》中的讲解。

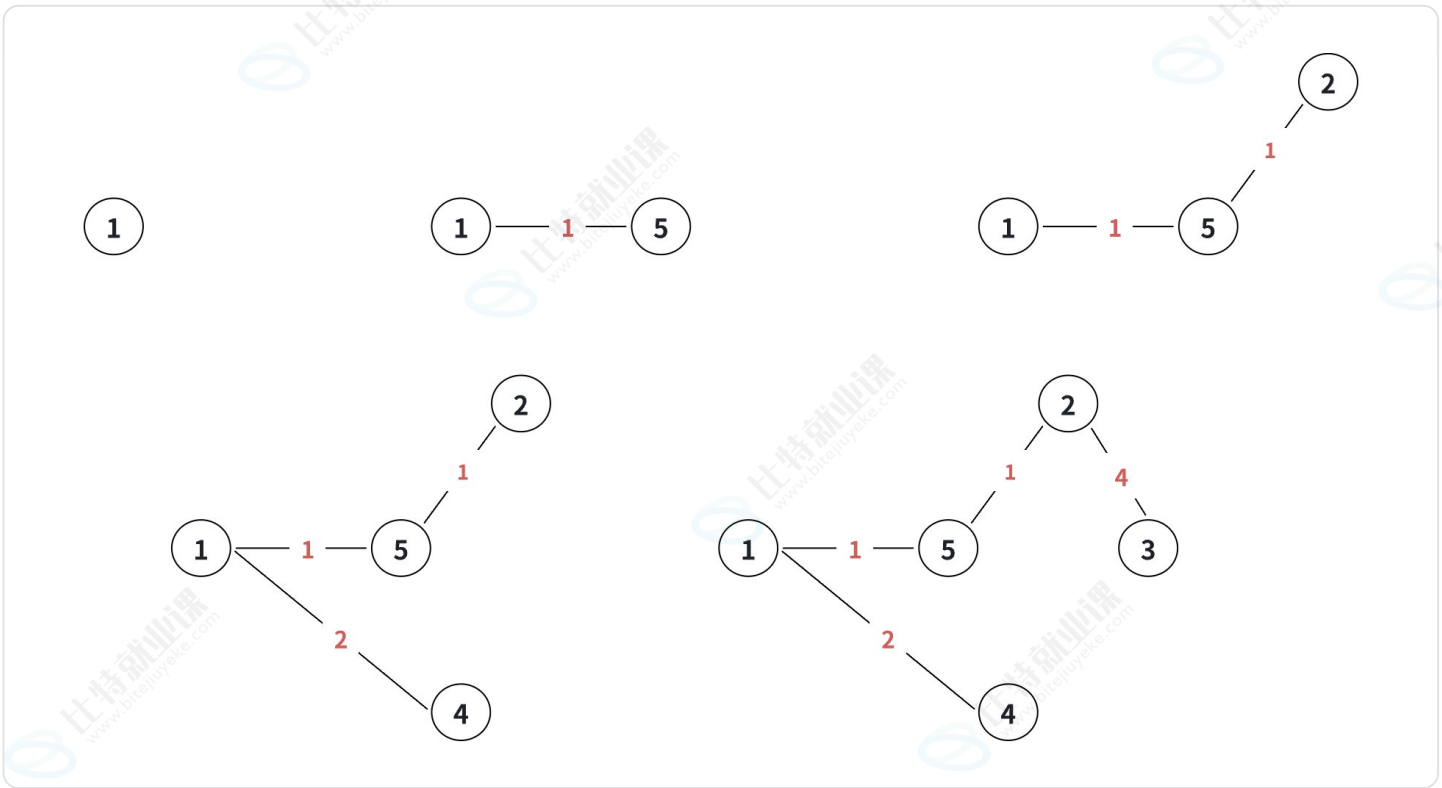
3.1 Prim 算法

核心：不断加点。

Prim 算法构造最小生成树的基本思想：

1. 从任意一个点开始构造最小生成树；
2. 将距离该树权值最小且不在树中的顶点，加入到生成树中。然后更新与该点相连的点到生成树的最短距离；
3. 重复 2 操作 n 次，直到所有顶点都加入为止。





代码实现 - 邻接矩阵：

```cpp
#include <iostream>
#include <cstring>

using namespace std;

const int N = 5010, INF = 0x3f3f3f3f;

int n, m;
int edges[N][N]; // 邻接矩阵存储图
```

```cpp

int dist[N]; // 某个点距离生成树的最短距离
bool st[N]; // 标记哪些点已经加入到生成树

int prim()
{
    // 初始化
    memset(dist, 0x3f, sizeof dist);
    dist[1] = 0;

    int ret = 0;
    for(int i = 1; i <= n; i++) // 循环加入 n 个点
    {
        // 1. 找最近点
        int t = 0;
        for(int j = 1; j <= n; j++)
            if(!st[j] && dist[j] < dist[t])
                t = j;

        // 判断是否联通
        if(dist[t] == INF) return INF;
        st[t] = true;
        ret += dist[t];

        // 2. 更新距离
        for(int j = 1; j <= n; j++) // 枚举 t 能走到哪
            dist[j] = min(dist[j], edges[t][j]);
    }

    return ret;
}

int main()
{
    cin >> n >> m;
    // 初始化 邻接矩阵
    memset(edges, 0x3f, sizeof edges);

    for(int i = 1; i <= m; i++)
    {
        int x, y, z; cin >> x >> y >> z;
        // 注意有重边的情况
        edges[x][y] = edges[y][x] = min(edges[x][y], z);
    }

    int ret = prim();
    if(ret == INF) cout << "orz" << endl;
```

```
57      else cout << ret << endl;
58
59      return 0;
60  }
```

代码实现 - 邻接表 - vector 数组：

```
1   #include <iostream>
2   #include <vector>
3   #include <cstring>
4
5   using namespace std;
6
7   typedef pair<int, int> PII;
8   const int N = 5010, INF = 0x3f3f3f3f;
9
10  int n, m;
11  vector<PII> edges[N];
12
13  int dist[N];
14  bool st[N];
15
16  int prim()
17  {
18      memset(dist, 0x3f, sizeof dist);
19      dist[1] = 0;
20      int ret = 0;
21
22      for(int i = 1; i <= n; i++)
23      {
24          // 1. 找最近点
25          int t = 0;
26          for(int j = 1; j <= n; j++)
27              if(!st[j] && dist[j] < dist[t])
28                  t = j;
29
30          // 判断是否联通
31          if(dist[t] == INF) return INF;
32          st[t] = true;
33          ret += dist[t];
34
35          // 2. 更新距离
36          for(auto& p : edges[t])
37          {
```

```
            int a = p.first, b = p.second;
            // t->a 权值是 b
            dist[a] = min(dist[a], b);
        }
    }

    return ret;
}

int main()
{
    cin >> n >> m;
    for(int i = 1; i <= m; i++)
    {
        int x, y, z; cin >> x >> y >> z;
        // 如果有重边，怎么办?
        edges[x].push_back({y, z});
        edges[y].push_back({x, z});
    }

    int ret = prim();
    if(ret == INF) cout << "orz" << endl;
    else cout << ret << endl;

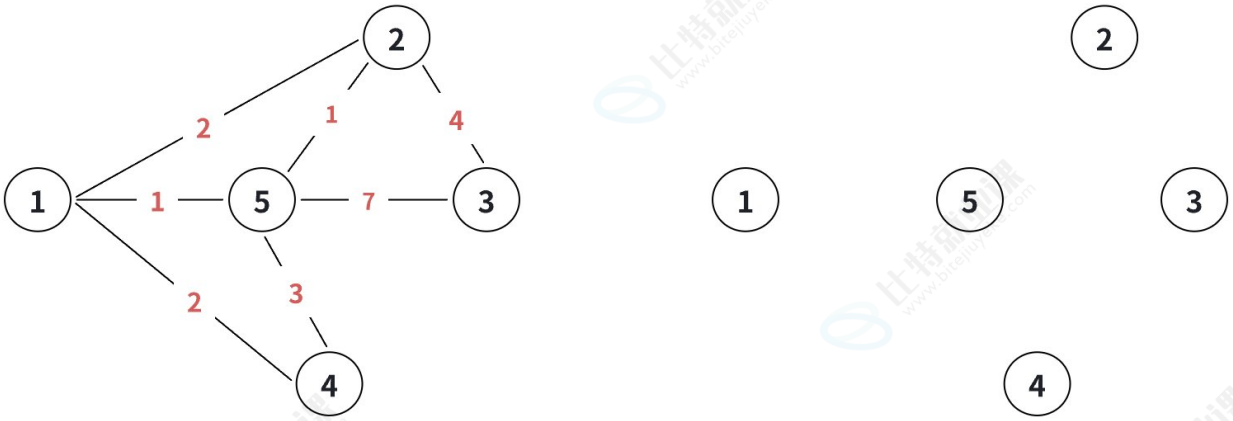
    return 0;
}
```

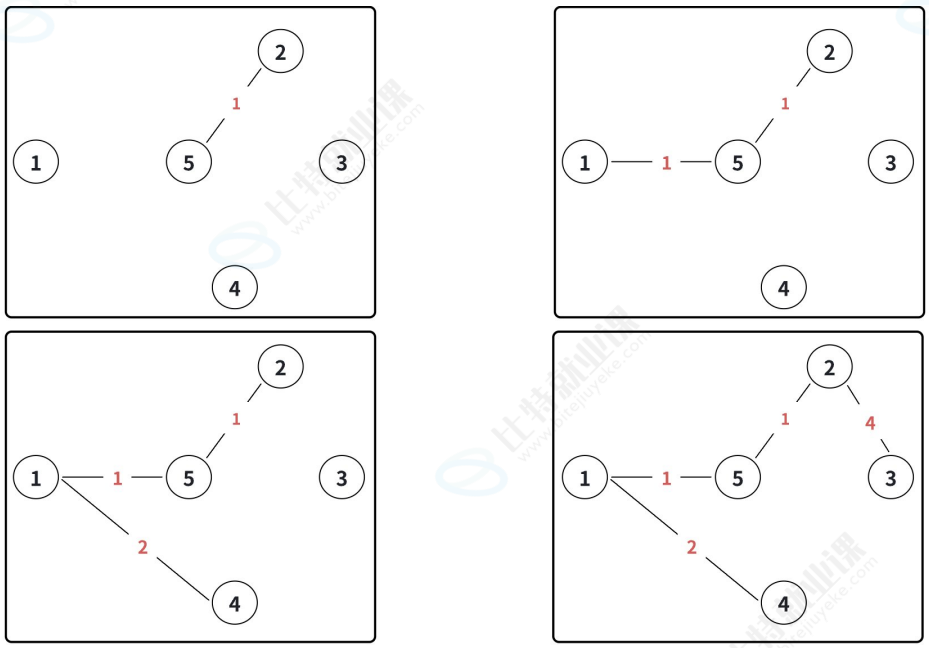
3.2 Kruskal 算法

核心：不断加边。

Kruskal 算法构造最小生成树的基本思想：

1. 所有边按照权值排序；
2. 每次选出权值最小且两端顶点不连通的一条边，直到所有顶点都联通。





代码实现：

```cpp
#include <iostream>
#include <algorithm>

using namespace std;

const int N = 5010, M = 2e5 + 10, INF = 0x3f3f3f3f;

int n, m;
struct node
{
```

```cpp
    int x, y, z;
}a[M];

int fa[N]; // 并查集

bool cmp(node& a, node& b)
{
    return a.z < b.z;
}

int find(int x)
{
    return x == fa[x] ? fa[x] : fa[x] = find(fa[x]);
}

int kk()
{
    sort(a + 1, a + 1 + m, cmp);

    int cnt = 0;
    int ret = 0;
    for(int i = 1; i <= m; i++)
    {
        int x = a[i].x, y = a[i].y, z = a[i].z;

        int fx = find(x), fy = find(y);
        if(fx != fy)
        {
            cnt++;
            ret += z;
            fa[fx] = fy;
        }
    }

    return cnt == n - 1 ? ret : INF;
}

int main()
{
    cin >> n >> m;
    for(int i = 1; i <= m; i++) cin >> a[i].x >> a[i].y >> a[i].z;

    // 初始化并查集
    for(int i = 1; i <= n; i++) fa[i] = i;

    int ret = kk();
    if(ret == INF) cout << "orz" << endl;
```

```
58        else cout << ret << endl;
59
60        return 0;
61    }
```

3.3 【模板】最小生成树

> **题目来源：** 洛谷
>
> **题目链接：** P3366 【模板】最小生成树
>
> **难度系数：** ★★

【题目描述】

> 如题，给出一个无向图，求出最小生成树，如果该图不连通，则输出 `orz`。

【输入描述】

> 第一行包含两个整数 N, M，表示该图共有 N 个结点和 M 条无向边。
>
> 接下来 M 行每行包含三个整数 X_i, Y_i, Z_i，表示有一条长度为 Z_i 的无向边连接结点 X_i, Y_i。

【输出描述】

> 如果该图连通，则输出一个整数表示最小生成树的各边的长度之和。如果该图不连通则输出 `orz`。

【示例一】

> 输入：
>
> ```
> 4 5
> 1 2 2
> 1 3 2
> 1 4 3
> 2 3 4
> 3 4 3
> ```
>
> 输出：
>
> ```
> 7
> ```

解法：

参考前两个板块里面的代码。

3.4 买礼物